

APRENDIZAJE DE MÁQUINAS (ACIF104) INFORME FASE 3 — SUMATIVA 2

Nombre integrante/s:

Javiera Chávez
Joaquín Olivares
Claudio Yáñez

Curso: Aprendizaje de Máquina

Fecha: 18-08-2026

Índice

Índice2

1. Introducción3

2. Problemática3

 2.1. Requisitos iniciales.....3

3. Metodología.....4

 3.1. Fases aplicadas hasta ahora4

 3.2. Plan de trabajo detallado4

4. Desarrollo del proyecto5

 4.a. Comparación de técnicas de Machine Learning y arquitecturas de Deep Learning.....5

 4.b. Requisitos funcionales y no funcionales5

 4.c. Arquitectura: justificación de la selección del modelo6

 4.d. Modelos: partición de datos y balanceo de clases7

 4.e. Sistema: avance en la integración frontend/backend8

5. Resultados.....9

 5.1. Análisis interpretativo mediante SHAP.....9

6. Propuesta de mejoras..... 10

 6.1. Limitaciones identificadas 10

 6.2. Mejoras propuestas..... 10

7. Código fuente en GitHub 10

8. Bibliografía 11

1. Introducción

Este informe corresponde a la tercera fase del proyecto Perfectime, desarrollado en el marco de la asignatura Aprendizaje de Máquina (ACIF104). Continúa el trabajo documentado en el informe de la Fase 2, profundizando en la validación y optimización de los modelos predictivos, ampliando la comparación de técnicas de Machine Learning y arquitecturas de Deep Learning, y avanzando en la integración entre el modelo entrenado y el sistema de software (frontend/backend).

El documento se organiza en las siguientes secciones: problemática y requisitos iniciales, metodología y plan de trabajo, desarrollo del proyecto (comparación de técnicas, requisitos, arquitectura del modelo, partición y balanceo de datos, y avance del sistema), resultados y su interpretación mediante SHAP, propuesta de mejoras, código fuente en GitHub y bibliografía en formato APA 7.^a edición.

2. Problemática

La transformación digital impulsada por la Industria 4.0 ha incrementado de manera significativa la cantidad de información disponible para las organizaciones. Los procesos industriales actuales incorporan sensores, dispositivos IoT y sistemas de monitoreo capaces de registrar continuamente variables como temperatura, velocidad de rotación, torque y desgaste de los equipos, lo que ha permitido avanzar hacia modelos de gestión más inteligentes, sustentados en el análisis de datos históricos y en tiempo real (Hamasha et al., 2025).

A pesar de estos avances, gran parte de las organizaciones continúa utilizando estrategias tradicionales de mantenimiento correctivo y preventivo, las cuales presentan limitaciones importantes: detenciones inesperadas, altos costos de reparación y reemplazos innecesarios de componentes que aún conservan vida útil (Benhanifia et al., 2025). Meitz et al. (2025) señalan que las actividades de mantenimiento pueden representar entre un 15 % y un 70 % de los costos totales de producción, dependiendo del tipo de industria.

El principal desafío técnico consiste en identificar oportunamente las condiciones que anteceden a una falla a partir del gran volumen de datos generado por los sensores, tarea que resulta impracticable de forma manual (Guidotti et al., 2025). En respuesta a esta problemática surge el mantenimiento predictivo, estrategia que utiliza datos históricos y variables de operación para estimar la probabilidad de falla de un equipo y planificar las intervenciones en el momento más oportuno, constituyéndose en una de las principales aplicaciones del aprendizaje automático dentro de la Industria 4.0 (Benhanifia et al., 2025; Hamasha et al., 2025).

La literatura también evidencia desafíos abiertos: Guidotti et al. (2025) destacan la escasez de conjuntos de datos industriales públicos y la necesidad de soluciones más interpretables; Aminzadeh et al. (2025) demuestran, mediante un caso práctico en compresores industriales, que el valor de estas soluciones depende tanto del algoritmo como de su integración en una plataforma funcional. Considerando estos antecedentes, este proyecto continúa el desarrollo de Perfectime, una plataforma de mantenimiento predictivo que integra técnicas de aprendizaje automático supervisado con una interfaz orientada a supervisores de mantenimiento.

2.1. Requisitos iniciales

A partir de la problemática descrita, esta fase mantiene y profundiza los requisitos definidos en la Fase 2 (ver sección 6.b), centrados en tres ejes: (1) ampliar y validar rigurosamente el núcleo predictivo, contrastando múltiples técnicas de Machine Learning y arquitecturas de Deep Learning antes de fijar el modelo de producción; (2) garantizar la trazabilidad de los datos y del entrenamiento (partición, balanceo, métricas) para que los

resultados sean reproducibles por terceros; y (3) avanzar en la integración real entre el modelo servido por la API y la interfaz de usuario, reemplazando progresivamente los datos simulados del prototipo visual de la Fase 2.

3. Metodología

El proyecto continúa desarrollándose bajo la metodología CRISP-DM (Cross Industry Standard Process for Data Mining; Chapman et al., 2000), la cual organiza el trabajo en seis etapas: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Las Fases 1 y 2 del proyecto cubrieron íntegramente las cuatro primeras etapas; esta Fase 3 profundiza en modelado y evaluación —ampliando la comparación de técnicas y optimizando hiperparámetros— y avanza en el despliegue, mediante la integración del modelo con la API REST y el frontend.

3.1. Fases aplicadas hasta ahora

- **Fase 1 (Comprensión del entorno):** análisis del problema de mantenimiento predictivo, revisión de literatura y definición de objetivos.
- **Fase 2 (Preparación de los datos):** análisis exploratorio, evaluación de calidad, limpieza, codificación y escalado del dataset AI4I 2020, además de un primer modelado con tres técnicas (Random Forest, XGBoost, MLP) y explicabilidad SHAP.
- **Fase 3 (Validación y optimización — este informe):** ampliación a tres técnicas de Machine Learning y tres arquitecturas de Deep Learning, comparación de tres estrategias de balanceo de clases, ajuste de hiperparámetros mediante GridSearchCV, reorganización del repositorio en carpetas estandarizadas (datasets, notebooks, modelos, backend, frontend) y avance en la integración frontend/backend.

3.2. Plan de trabajo detallado

El siguiente plan distribuye las tareas pendientes para el cierre de esta fase y el inicio de la Sumativa 3. Los responsables se mantienen como una propuesta de distribución de carga; el equipo puede reasignarlos según disponibilidad.

Tarea	Responsable(s)	Plazo
Ampliación del notebook: 3.ª técnica ML (Regresión Logística) y 3 arquitecturas DL (MLP superficial, profunda, ancha)	Claudio Yáñez	11–14 ago 2026
Reorganización del repositorio (datasets/notebooks/modelos) y corrección de rutas del pipeline	Claudio Yáñez	13–14 ago 2026
Redacción del informe Fase 3 (problemática, metodología, resultados, mejoras)	Javiera Chávez, Claudio Yáñez	13–17 ago 2026
Revisión de bibliografía adicional y formato APA 7	Javiera Chávez	15–17 ago 2026
Conexión de "Carga de archivo CSV" y "Explicabilidad SHAP" del frontend a la API real	Joaquín Olivares	18–24 ago 2026
Pruebas de extremo a extremo del sistema (backend + frontend) y ajustes finales	Equipo completo	25–28 ago 2026

Tabla 1. Plan de trabajo — Fase 3.

4. Desarrollo del proyecto

4.a. Comparación de técnicas de Machine Learning y arquitecturas de Deep Learning

Con el objetivo de seleccionar el modelo más adecuado para el núcleo predictivo de Perfectime, se entrenaron y compararon seis modelos sobre el mismo conjunto de entrenamiento balanceado con SMOTE y evaluados sobre el mismo conjunto de prueba (3.000 registros, nunca utilizado durante el entrenamiento): tres técnicas de Machine Learning clásico y tres arquitecturas de Deep Learning (variantes de Perceptrón Multicapa).

Técnica	Tipo	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	ML — Ensemble (Bagging)	0.9627	0.4679	0.7157	0.5659	0.9713
XGBoost	ML — Ensemble (Boosting)	0.9753	0.6077	0.7745	0.6810	0.9780
Regresión Logística	ML — Lineal	0.8353	0.1475	0.8039	0.2492	0.8888
MLP superficial (DL-A)	DL — 1 capa (16 neuronas)	0.9313	0.3169	0.8824	0.4663	0.9692
MLP profunda (DL-B)	DL — 3 capas (64-32-16)	0.9660	0.5000	0.7255	0.5920	0.9492
MLP ancha (DL-C)	DL — 1 capa (128 neuronas)	0.9580	0.4388	0.8431	0.5772	0.9744

Tabla 2. Comparación de las 3 técnicas de ML y las 3 arquitecturas de DL sobre el conjunto de prueba (resultados obtenidos en notebooks/Sumativa.ipynb, sección 8.2).

XGBoost obtiene el mejor desempeño global (AUC-ROC 0,9780; F1-Score 0,6810), seguido de cerca por dos de las arquitecturas de Deep Learning: la MLP ancha (AUC 0,9744) y la MLP superficial (AUC 0,9692). Un hallazgo relevante es que la MLP profunda —con más capas y parámetros— obtuvo un desempeño inferior a las otras dos arquitecturas neuronales (AUC 0,9492, el más bajo entre las tres DL). Esto es consistente con la literatura: en problemas tabulares de baja dimensionalidad como este (7 variables predictoras), aumentar la profundidad de la red no necesariamente mejora la capacidad de representación y puede, en cambio, dificultar la optimización con un volumen de datos moderado (Goodfellow et al., 2016).

La Regresión Logística, al ser un modelo lineal, obtuvo el desempeño más bajo (AUC 0,8888): su Recall es alto (0,8039) pero su Precision es muy baja (0,1475), ya que al no poder trazar una frontera de decisión no lineal necesita clasificar como "riesgo" a muchos registros para capturar la mayoría de las fallas reales, generando numerosas falsas alarmas. Este resultado confirma empíricamente la necesidad de modelos no lineales (ensambles de árboles o redes neuronales) para este problema.

4.b. Requisitos funcionales y no funcionales

Se mantienen los requisitos definidos en la Fase 2, actualizados para reflejar el estado actual de integración del sistema.

Requisitos funcionales

ID	Requisito	Estado
RF-01	Carga de archivos CSV por lotes vía POST /predict-batch	Implementado en backend; pendiente conexión con frontend
RF-02	Validación de estructura y rangos de los datos mediante esquemas Pydantic	Implementado

ID	Requisito	Estado
RF-03	Preprocesamiento idéntico a entrenamiento (One-Hot + StandardScaler) en la inferencia	Implementado
RF-04	Estimación de probabilidad de falla mediante el modelo XGBoost servido	Implementado y conectado al frontend (predicción individual)
RF-05	Desglose de variables mediante SHAP en cada predicción	Implementado en backend; pantalla de explicabilidad aún simulada
RF-06	Presentación de resultados mediante indicador de riesgo y probabilidad	Implementado (predicción individual)
RF-07	Historial de predicciones con filtros	Pendiente (frontend con datos simulados)
RF-08	Exportación de resultados en CSV	Pendiente
RF-09	Endpoint de monitoreo (GET /health)	Implementado

Tabla 3. Requisitos funcionales y su estado de implementación al cierre de la Fase 3.

Requisitos no funcionales

Categoría	Requisito
Usabilidad	Interfaz intuitiva que permita interpretar fácilmente las predicciones.
Explicabilidad	Mostrar las variables que más influyen en la predicción cuando sea técnicamente posible (SHAP).
Escalabilidad	Arquitectura modular que permita incorporar nuevos conjuntos de datos y modelos sin modificar la estructura principal.
Confiabilidad	Resultados consistentes a partir de datos previamente validados; verificación de integridad del modelo serializado tras cada reentrenamiento.
Monitoreabilidad	Registro de predicciones, archivos procesados y disponibilidad del modelo mediante el endpoint /health.
Mantenibilidad	Estructura de carpetas estandarizada (datasets, notebooks, modelos, backend, frontend) y rutas del pipeline independientes del directorio de ejecución.
Portabilidad	Entorno reproducible mediante environment.yml (Conda) documentado en el README.

Tabla 4. Requisitos no funcionales.

4.c. Arquitectura: justificación de la selección del modelo

La arquitectura final de Perfectime utiliza XGBoost como modelo de producción, por ser la técnica con mejor equilibrio entre desempeño (AUC-ROC 0,9780), capacidad de manejar el desbalance de clases y explicabilidad vía SHAP (TreeExplainer). No obstante, la decisión se fundamenta en la comparación sistemática contra las otras cinco técnicas, incluyendo tres arquitecturas de Deep Learning cuyo diseño se detalla a continuación.

Arquitectura	Capas ocultas	Neuronas por capa	Activación	Optimizador	AUC-ROC
A — Superficial	1	16	ReLU	Adam	0.9692
B — Profunda	3	64, 32, 16	ReLU	Adam	0.9492
C — Ancha	1	128	ReLU	Adam	0.9744

Tabla 5. Configuración de las tres arquitecturas de Deep Learning comparadas (MLPClassifier, scikit-learn).

Las tres arquitecturas comparten la función de activación ReLU en las capas ocultas y el optimizador Adam, variando únicamente la profundidad (número de capas) y el ancho (neuronas por capa), lo que permite aislar el efecto de la capacidad estructural de la red sobre el desempeño. Los resultados (Tabla 2) muestran que la arquitectura ancha (una capa de 128 neuronas) supera a la arquitectura profunda (tres capas de 64-32-16 neuronas), evidenciando que, para las siete variables predictoras de este problema, una mayor capacidad concentrada en una sola capa generaliza mejor que distribuir la misma capacidad en múltiples capas — una arquitectura más profunda introduce más parámetros a optimizar sin una ganancia proporcional de capacidad representacional, dado el tamaño relativamente acotado del conjunto de entrenamiento (13.526 registros tras SMOTE).

Adicionalmente, se ajustaron los hiperparámetros de XGBoost mediante GridSearchCV (108 combinaciones $\times$ 5 folds de validación cruzada estratificada, optimizando F1-Score de la clase positiva). La configuración óptima resultó en `n_estimators=300`, `max_depth=7`, `learning_rate=0.2`, `subsample=0.8`, `colsample_bytree=1.0`, con una mejora marginal sobre el conjunto de prueba (AUC-ROC de 0,9780 a 0,9796; F1-Score de 0,6810 a 0,6840), lo que confirma que el modelo base ya operaba cerca de su capacidad óptima para este problema.

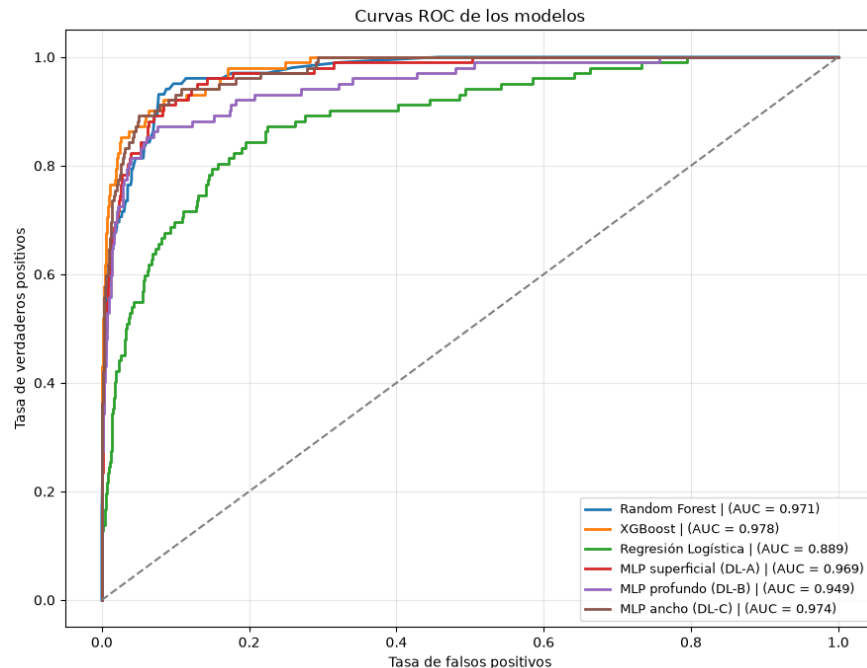


Figura 1. Curvas ROC de las 6 técnicas comparadas.

4.d. Modelos: partición de datos y balanceo de clases

El conjunto de datos (10.000 registros, 7 variables predictoras tras el preprocesamiento) se dividió en 70 % entrenamiento (7.000 registros) y 30 % prueba (3.000 registros), utilizando muestreo estratificado (parámetro `stratify`) para conservar la proporción original de la variable objetivo (3,39 % de casos con falla) en ambas particiones.

Dado el fuerte desbalance de clases, se compararon tres técnicas de balanceo/muestreo sobre el conjunto de entrenamiento, manteniendo fijo el conjunto de prueba, el modelo (XGBoost) y la semilla aleatoria:

Escenario	N° entrenamiento	% clase positiva	Accuracy	Precision	Recall	F1-Score	AUC-ROC
1. Sin balanceo	7.000	3.39 %	0.9833	0.8250	0.6471	0.7253	0.9793
2. Submuestreo (Undersampling)	474	50.00 %	0.9037	0.2533	0.9412	0.3992	0.9655
3. SMOTE (Oversampling sintético)	13.526	50.00 %	0.9753	0.6077	0.7745	0.6810	0.9780

Tabla 6. Comparación de tres técnicas de balanceo de clases (XGBoost, notebook sección 8.3).

El escenario sin balanceo maximiza Precision y Accuracy, pero a costa del Recall: al estar dominado por la clase mayoritaria, el modelo tiende a no detectar una parte importante de las fallas reales — el peor escenario posible en mantenimiento predictivo, donde cada falso negativo equivale a una detención no anticipada. El submuestreo maximiza el Recall (0,9412), pero descarta la mayor parte de los registros de la clase mayoritaria (de 6.763 a 237), degradando severamente la Precision por el reducido volumen de entrenamiento resultante. SMOTE ofrece el mejor equilibrio general (F1-Score más alto entre las tres estrategias, 0,6810) sin sacrificar el AUC-ROC, por lo que se mantiene como la técnica de balanceo del modelo de producción.

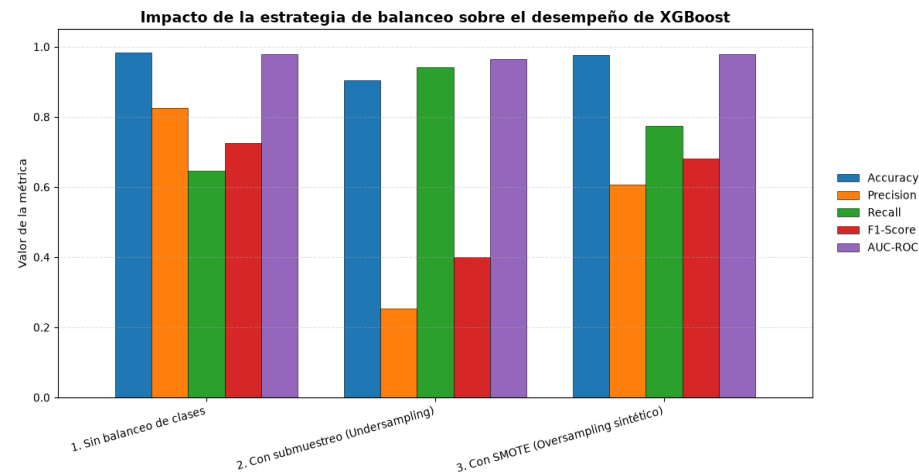


Figura 2. Impacto de la estrategia de balanceo sobre el desempeño de XGBoost.

4.e. Sistema: avance en la integración frontend/backend

Durante esta fase se construyó desde cero el servicio FastAPI (backend/main.py) que reemplaza la lógica simulada del prototipo visual entregado en la Fase 2. El backend carga el modelo XGBoost, el StandardScaler y el orden de columnas al iniciar, y expone los endpoints POST /predict (predicción individual con desglose SHAP), POST /predict-batch (predicción por lotes) y GET /health (estado del servicio).

La pantalla de "Predicción individual" del frontend (frontend/index_1.html) fue conectada de extremo a extremo a la API real: el formulario fue actualizado para solicitar las cinco variables reales del dataset (antes pedía variables simuladas que el modelo nunca vio, como vibración o humedad) y la función de predicción reemplaza la fórmula de riesgo simulada por una llamada fetch al endpoint /predict. Las pantallas de "Carga de archivo CSV", "Explicabilidad SHAP", "Monitoreo" e "Historial" continúan utilizando datos simulados y quedan planificadas para la siguiente iteración (ver Tabla 1), aunque sus endpoints de backend correspondientes (/predict-batch, /health) ya se encuentran disponibles.

5. Resultados

El modelo seleccionado (XGBoost, optimizado mediante GridSearchCV) fue evaluado sobre el conjunto de prueba de 3.000 registros, no utilizado durante el entrenamiento ni el balanceo de clases.

Métrica	XGBoost (línea base)	XGBoost (optimizado)
Accuracy	0.9753	0.9757
Precision (clase Con falla)	0.6077	0.6124
Recall (clase Con falla)	0.7745	0.7745
F1-Score (clase Con falla)	0.6810	0.6840
AUC-ROC	0.9780	0.9796

Tabla 7. Métricas del modelo final sobre datos no vistos.

La matriz de confusión del modelo XGBoost muestra 2.847 verdaderos negativos, 79 verdaderos positivos, 51 falsos positivos y 23 falsos negativos, evidenciando una buena capacidad de detección de fallas reales con un número acotado de falsas alarmas.

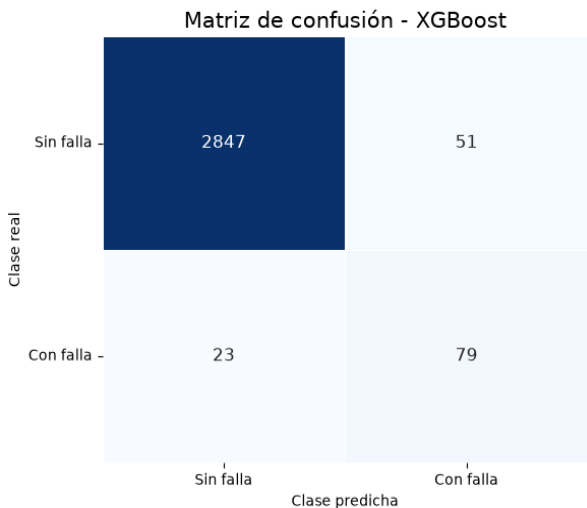


Figura 3. Matriz de confusión — XGBoost.

5.1. Análisis interpretativo mediante SHAP

Se aplicó la técnica SHAP (SHapley Additive exPlanations), mediante TreeExplainer sobre el modelo XGBoost, para analizar la contribución de cada variable a las predicciones del modelo. El gráfico de importancia global (SHAP Summary Plot) muestra que Tool_wear_min (desgaste de la herramienta), Torque_Nm y Rotational_speed_rpm son las variables con mayor impacto en las predicciones, consistente con el conocimiento del dominio: estas tres variables están directamente asociadas al estrés mecánico acumulado del equipo. Las variables de temperatura presentan una influencia intermedia, mientras que las variables categóricas del tipo de producto (Type_L, Type_M) tienen la menor contribución relativa.

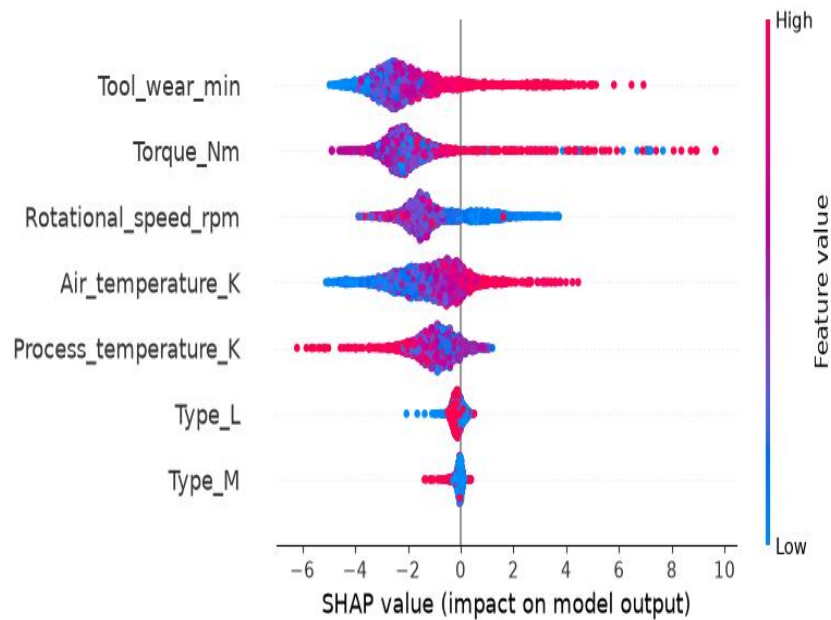


Figura 4. SHAP Summary Plot — importancia global de las variables (modelo XGBoost).

6. Propuesta de mejoras

6.1. Limitaciones identificadas

- **Naturaleza estática de los datos tabulares:** el dataset AI4I 2020 entrega mediciones en instantes discretos, lo que limita la capacidad de los modelos actuales para capturar la evolución temporal de la degradación de una componente previa a una falla por fatiga o desgaste.
- **Sensibilidad al desbalanceo severo en tipos de falla infrecuentes:** aunque SMOTE mejora el balance agregado de la clase "con falla", tipos de falla específicos y extremadamente infrecuentes dentro de esa clase (p. ej., fallas aleatorias) quedan subrepresentados incluso tras el remuestreo, aumentando el riesgo de falsos negativos en escenarios críticos.

6.2. Mejoras propuestas

Mejora 1 — Ventanas temporales deslizantes y arquitectura híbrida CNN-LSTM: transformar la ingesta de telemetría en ventanas secuenciales (agrupando los últimos minutos de operación) y entrenar una arquitectura híbrida 1D-CNN + LSTM sobre esa estructura. La capa convolucional extraería patrones de fluctuación local en torque y temperatura, mientras que la celda LSTM modelaría la acumulación paulatina de estrés mecánico, permitiendo estimar no solo si ocurrirá una falla, sino la Vida Útil Restante (RUL) del equipo. Esta mejora responde directamente a la limitación de datos estáticos identificada arriba.

Mejora 2 — Aprendizaje activo y monitoreo de deriva de datos (data drift): integrar un módulo que identifique automáticamente las predicciones con menor margen de certidumbre (p. ej., probabilidades entre 0,45 y 0,55) y las envíe a la interfaz del supervisor para validación humana, retroalimentando y reentrenando periódicamente el modelo. En entornos industriales reales las condiciones operativas cambian constantemente (concept drift), degradando la precisión del modelo con el tiempo; esta mejora reduce los falsos positivos/negativos y asegura la adaptabilidad continua del sistema.

7. Código fuente en GitHub

El código fuente completo del proyecto —notebook, backend, frontend, datasets y modelos serializados— se encuentra disponible públicamente en el siguiente repositorio: https://github.com/XetuRiot/Acif104_Grupo5.

Durante esta fase el repositorio fue reorganizado en carpetas estandarizadas (datasets/, notebooks/, modelos/, backend/, frontend/, Documentacion/), y las rutas del pipeline (lectura del dataset, guardado de artefactos del modelo) se hicieron independientes del directorio desde el cual se ejecute Jupyter, corrigiendo un error de ruta relativa identificado durante la revisión de esta fase (ver README.md, sección "Registro de implementación").

El README.md del repositorio documenta las instrucciones de instalación, configuración del entorno Conda y ejecución del sistema completo (notebook, backend y frontend), garantizando la reproducibilidad del proyecto.

8. Bibliografía

- Aminzadeh, A., Sattarpanah Karganroudi, S., Majidi, S., Dabompre, C., Azaiez, K., Mitride, C., & Sénéchal, E. (2025). A machine learning implementation to predictive maintenance and monitoring of industrial compressors. *Sensors*, 25(4), Artículo 1006. <https://doi.org/10.3390/s25041006>
- Benhanifia, A., Ben Cheikh, Z., Oliveira, P. M., Valente, A., & Lima, J. (2025). Systematic review of predictive maintenance practices in the manufacturing sector. *Intelligent Systems with Applications*, 26, Artículo 200501. <https://doi.org/10.1016/j.iswa.2025.200501>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Guidotti, D., Pandolfo, L., & Pulina, L. (2025). A systematic literature review of supervised machine learning techniques for predictive maintenance in Industry 4.0. *IEEE Access*, 13, 102479–102504. <https://doi.org/10.1109/ACCESS.2025.3578686>
- Hamasha, M. M., Albedoor, Q., Hamasha, S., Ali, H., Qamar, A., & Berrah, F. (2025). A comprehensive framework for IoT-driven predictive maintenance: Leveraging AI and edge computing for enhanced equipment reliability. *Journal of Applied Engineering Science*, 23(3), 471–486. <https://doi.org/10.5937/jaes0-57002>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.
- Meitz, L., Senge, J., Wagenhals, T., Schöler, T., Hähner, J., Edinger, J., & Krupitzer, C. (2025). A literature review framework and open research challenges for predictive maintenance in Industry 4.0. *Computers & Industrial Engineering*, 206, Artículo 111193. <https://doi.org/10.1016/j.cie.2025.111193>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.